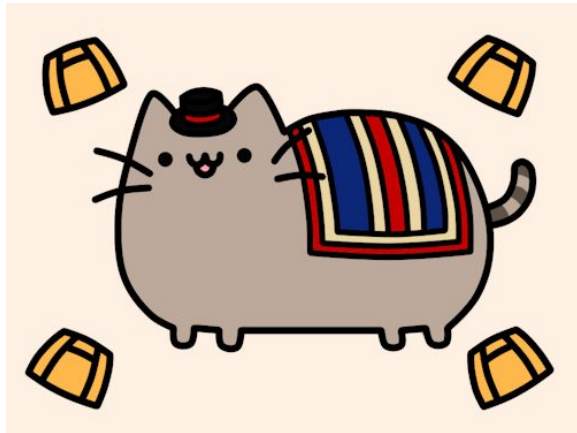


Programación Avanzada
IIC2233 2026-2

Anuncios

3 de septiembre de 2026



- Hoy tendremos nuestra segunda actividad evaluada.
 - Suban la actividad a la carpeta **Actividades/AC2/**
 - Recuerden **marcar su salida con la TUC**, sino quedarán como ausentes.
 - La próxima semana tendremos la **11** en **horario de cátedra**. Aprovechen el Ejercicio Propuesto 1 para estudiar.
-

Control de Entrada



git add, git commit, git push

Comentarios AC 2

- Suban la actividad a la carpeta **Actividades/AC2/**
- Recuerden que esto es una **actividad individual**. **Mantengan el silencio.**
- **def __init__(self, valor: Caja | Registro) -> None:**
Significa que valor puede ser Caja o Registro
- Si su código queda en un **loop infinito**, usen “**Ctrl + C**” para detener la ejecución del programa.

Cierre

Actividad 2: Excepciones, Iterables y Iteradores



Parte 1: Agregar caja

← significa “asignar o guardar un valor”

agregar(valor):

nuevo ← nuevo Nodo que guarda valor

si cabeza es nulo:

 cabeza ← nuevo

 cola ← nuevo

si no:

 cola.siguiente ← nuevo

 cola ← nuevo

largo ← largo + 1

Parte 1: Retirar caja

← significa “asignar o guardar un valor”

```
retirar_primera(self):  
    si cabeza es nulo:  
        levantar IndexError  
  
    nodo_a_retirar ← cabeza  
    cabeza          ← nodo_a_retirar.siguiente  
    si cabeza es nulo:  
        cola ← nulo  
  
    largo          ← largo - 1  
    retornar nodo_a_retirar.valor
```

Parte 2: Iterable e iterador

```
ListaLigada.__iter__():  
    # Se ejecuta al usar iter(lista).  
    # Crea un recorrido independiente.  
    retornar IteradorListaLigada(cabeza)
```

Parte 2: Iterable e iterador

```
IteradorListaLigada.__init__(cabeza):  
    # Guarda la posición inicial del recorrido.  
    nodo_actual ← cabeza
```

```
IteradorListaLigada.__iter__():  
    # Se ejecuta al usar iter(iterador).  
    retornar self
```


Parte 2: Iterable e iterador

```
IteradorListaLigada.__next__():  
    # Se ejecuta al usar next(iterador).  
    si nodo_actual es nulo:  
        levantar StopIteration  
  
    valor ← nodo_actual.valor  
    nodo_actual ← nodo_actual.siguiente  
  
    retornar valor
```

Parte 3: ¿Qué hacer ante una caja inválida?

Si una máquina detecta que una caja es inválida,
¿qué alternativa permite comunicarlo correctamente?

- A)** Mostrar un mensaje con `print`
- B)** Retornar el mensaje
- C)** Levantar una excepción

Parte 3: ¿Qué hacer ante una caja inválida?

Si una máquina detecta que una caja es inválida, ¿qué alternativa permite comunicarlo correctamente?

- A) Mostrar un mensaje con `print` B) Retornar el mensaje **C) Levantar una excepción**

```
inspeccionar(caja):  
    si el código no es válido:  
        levantar CajaRechazadaError("código inválido")  
<<  -- continua -->
```

Parte 4: Generadores, ¿corchetes o paréntesis?

Si queremos retornar un generador, ¿cuál alternativa sirve?

A) `registros_por_resultado(bitacora, resultado):`
 retornar `[r para cada r en bitacora si r.resultado = res]`

B) `registros_por_resultado(bitacora, resultado):`
 retornar `(r para cada r en bitacora si r.resultado = res)`

Parte 4: Generadores, ¿corchetes o paréntesis?

Si queremos retornar un generador, ¿cuál alternativa sirve?

A) `registros_por_resultado(bitacora, resultado):`
 retornar `[r para cada r en bitacora si r.resultado = res]`

B) `registros_por_resultado(bitacora, resultado):`
 retornar `(r para cada r en bitacora si r.resultado = res)`